



CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir



Access Specifiers

Public, Private and Protected



Member Access

Each member in a class has a property called an **access level**.
This determines who can access that member.

C++ has three different access levels.

- Public

- Private

- Protected (*will talk about during inheritance*)

Public Members



Accessible anywhere outside the class.

Accessible within the same class (by the member functions).

Accessible within other classes.

*Accessible within other functions in the global scope (including **main()**).*

But always, these public members will be accessed through an object.

```
1  #include<iostream>
2
3  class Position
4  {
5  public:
6
7      void showPosition()
8      {
9          std::cout << m_posX << ", " << m_posY << '\n';
10     }
11
12     int m_posX;
13     int m_posY;
14 };
15
16
17 int main()
18 {
19     Position pos1;
20
21     pos1.showPosition();
22
23     return 0;
24 }
```

The public members of the class (attributes or methods even) can be accessed **within the same class**.

This example also shows that public members are accessible in other functions, outside the class.

showPosition() is a public member being called from **main()**.

Note

Even though the attributes come after methods, I can access those same attributes in the methods. Members can be defined in any order within the class definition.

```
3  class Position
4  {
5  public:
6
7      void showPosition()
8      {
9          std::cout << m_posX << ", " << m_posY << '\n';
10     }
11
12     int m_posX;
13     int m_posY;
14 };
15
16 class Robot
17 {
18 public:
19
20     void moveRobot()
21     {
22         m_robotPos.m_posX += 5;
23         m_robotPos.m_posY += 10;
24     }
25
26     Position m_robotPos;
27 };
```

*The public members of class Position can be accessed **within a different class**.*

Private Members

(Default Access Level in Classes)



Members of a class are **private by default**.

That is, if you don't specify any access specifier, members will be considered private.

Private members are members in a class that are only accessible by other members of the **same class**.

```
3  class Position
4  {
5      // Note that there is no access specifier (public or private)
6
7      void showPosition()
8      {
9          std::cout << m_posX << ", " << m_posY << '\n';
10     }
11
12     int m_posX;
13     int m_posY;
14 };
```

*All the three members are private **by default**.*

*Note that members are accessible within their own class (e.g. **m_posX** and **m_posY**).*

*But **showPosition()** is not available anywhere outside the class.*

```
17  class Robot
18  {
19  public:
20
21      void showRobotPos()
22      {
23          m_robotPos.showPosition();
24      }
25
26      Position m_robotPos;
27 };
```

```
29  int main()
30  {
31      Position pos1;
32
33      pos1.showPosition();
34
35      return 0;
36 }
```


Having multiple access specifiers in your class is possible.

```
3  class Position
4  {
5  // All members here will be considered private by default.
6
7
8
9
10
11
12
13
14
15
16 };
```

Having multiple access specifiers in your class is possible.

```
3  class Position
4  {
5  // All members here will be considered private by default.
6  public:
7      void showPosition()
8      {
9          std::cout << m_posX << ", " << m_posY << '\n';
10     }
11 // All members till here will be considered public.
12
13
14
15
16 };
```

Having multiple access specifiers in your class is possible.

```
3  class Position
4  {
5  // All members here will be considered private by default.
6  public:
7      void showPosition()
8      {
9          std::cout << m_posX << ", " << m_posY << '\n';
10     }
11 // All members till here will be considered public.
12 private:
13     int m_posX;
14     int m_posY;
15 // All members till here will be considered private.
16 };
```

Best practice

Classes should generally make member variables private (or protected), and member functions public.

Access Levels Work on Per-Class Basis



Another way to read the title above would be that,

Access levels **do not** work on a **per-object basis**.

What does this mean?

```
1  #include<iostream>
2  #include<cmath>
3
4  class Robot
5  {
6  public:
7      double distanceFromNeighbor(const Robot& rob)
8      {
9
10
11
12
13
14
15      }
16
17  private:
18      int m_posX;
19      int m_posY;
20  };
21
22
23  int main()
24  {
25      Robot r1;
26      Robot r2;
27
28      std::cout << r1.distanceFromNeighbor(r2);
29
30      return 0;
31  }
```

*The method is being called through **r1** (so **r1** is implicitly being passed to the method).*

***r2** is passed as an argument to the method.*

We can say that it's passed explicitly.



```
1  #include<iostream>
2  #include<cmath>
3
4  class Robot
5  {
6  public:
7      double distanceFromNeighbor(const Robot& rob)
8      {
9          // [Redacted]
10
11
12
13
14
15     }
16
17 private:
18     int m_posX;
19     int m_posY;
20 };
21
22
23 int main()
24 {
25     Robot r1;
26     Robot r2;
27
28     std::cout << r1.distanceFromNeighbor(r2);
29
30     return 0;
31 }
```

Note the public method in class Robot.

*It accepts an object of this same class as a **const** reference.*

*Passing by reference helps prevent creating a copy of it. **const** ensures that this data will not accidentally be modified by our function.*

```
1  #include<iostream>
2  #include<cmath>
3
4  class Robot
5  {
6  public:
7      double distanceFromNeighbor(const Robot& rob)
8      {
9
10
11
12
13
14
15     }
16
17 private:
18     int m_posX;
19     int m_posY;
20 };
21
22
23 int main()
24 {
25     Robot r1;
26     Robot r2;
27
28     std::cout << r1.distanceFromNeighbor(r2);
29
30     return 0;
31 }
```

So within this function, what if I write,

`std::cout << m_posX;`

`std::cout << rob.m_posX;`

*Btw, the access level of **m_posX** is private.*


```
1  #include<iostream>
2  #include<cmath>
3
4  class Robot
5  {
6  public:
7      double distanceFromNeighbor(const Robot& rob)
8      {
9          int deltaX{ m_posX - rob.m_posX };
10         int deltaY{ m_posY - rob.m_posY };
11
12         double sqDistance{ std::pow(deltaX, 2) + std::pow(deltaY, 2) };
13
14         return std::sqrt(sqDistance);
15     }
16
17 private:
18     int m_posX;
19     int m_posY;
20 };
21
22
23 int main()
24 {
25     Robot r1;
26     Robot r2;
27
28     std::cout << r1.distanceFromNeighbor(r2);
29
30     return 0;
31 }
```

So within this function, what if I write,

`std::cout << m_posX;`

`std::cout << rob.m_posX;`

*Btw, the access level of **m_posX** is private.*

Both statements will work as intended.

*Access levels **do not** work on a per-object basis. They work on a per-class basis.*

Access Levels Work on Per-Class Basis



A member function can directly access private members (of the implicit object).

However, because **access levels are per-class**, not per-object, a member function can also directly access the private members of **ANY** other object of the same class type that is in scope.

Getters and Setters

Also called Accessors and Mutators, respectively.



Getters / Setters

Getters (or accessors) are methods used to get (or access) values of private attributes.

Setters (or mutators) are methods used to set (or mutate/change) values of private attributes.

They're just regular methods; nothing really unique about them.

```
1  #include <iostream>
2
3  class Position
4  {
5  private:
6      int m_xPos;
7      int m_yPos;
8      int m_zPos;
9
10 public:
11     void print()
12     {
13         std::cout << '(' << m_xPos << ", " << m_yPos << ", " << m_zPos << ") \n";
14     }
15
16     int getXPos() { return m_xPos; }           // getter for xPos
17     void setXPos(int xPos) { m_xPos = xPos; } // setter for xPos
18
19     int getYPos() { return m_yPos; }           // getter for yPos
20     void setYPos(int yPos) { m_yPos = yPos; } // setter for yPos
21
22     int getZPos() { return m_zPos; }           // getter for zPos
23     void setZPos(int zPos) { m_zPos = zPos; } // setter for zPos
24 };
25
26 int main()
27 {
28     Position d;
29     d.setXPos(-15);
30     std::cout << "xPos is: " << d.getXPos() << '\n';
31
32     return 0;
33 }
```

Getters and setters.

Adding “m_” as a prefix to your attribute names is helpful.

Otherwise, you may run the risk of having your local variables of the same name as your attributes.

This will result in scoping issues.



Why Getters and Setters?

What is the point of private when I'm creating public functions to read and update the private members?

One reason is to not burden the user of your class (other programmers) to understand your **entire** class.

They should only need to understand the public interface of your class in order to use your class.

Recall how you used Python's list class without actually knowing what the backend data structure is, what are the various private attributes of the class, etc.



Why Getters and Setters?

Another reason is related to **class invariants**.

Consider this **Fraction** class.

I will create objects which will simulate or model fractions.

Recall that the denominator of a fraction should not be zero (they become undefined otherwise).

This is a problem, and this can happen if my data is public.

```
#include <iostream>

class Fraction
{
public:
    int m_num;
    int m_den;

    void print()
    {
        std::cout << "Fraction(" << m_num << ", "
                    << m_den << ")\n";
    }
};
```

```
int main()
{
    Fraction f;
    f.m_den = 0;

    return 0;
}
```


If I have private members, direct access will not work.

*In this case, I have chosen to define just a getter. So **m_den** is read-only through the getter.*

Hence, I have more control if I have my attribute set to private.

```
class Fraction
{
private:
    int m_num;
    int m_den;

public:
    int getDen() const
    {
        return m_den;
    }
};
```

```
int main()
{
    Fraction f;
    f.m_den = 0;
    return 0;
}
```

*I can also define a setter, but notice that I'm not blindly changing values. Because a setter is just a function, I can add conditions to verify whether I'm making an appropriate change or not within the setter; I'm maintaining a **class invariant**.*

```
class Fraction
{
private:
    int m_num;
    int m_den;

public:
    int getDen() const
    {
        return m_den;
    }
};
```

```
void setDen(int newDen)
{
    if (newDen == 0)
    {
        // Raise some error and exit function
    }
    else
    {
        m_den = newDen;
    }
}
```

```
};
```

```
int main()
{
    Fraction f;

    return 0;
}
```

const Methods

What are **const** Methods?

Methods that are **const**, guarantee that they will not change the attributes of the implicit object.

Implicit object is the object that was used to call the method.

Good practice: if we know that a method is not meant to modify the implicit object, we make that method a **const**.

Should getters and setters be **const**?

Getters – *yes*.

Setters – *no*.

```
int getXPos() const
{
    return m_xPos;
}
```



```
3 class Position
4 {
5 private:
6     int m_xPos;
7     int m_yPos;
8     int m_zPos;
9
10 public:
11     int getXPos() const
12     {
13         m_xPos = 54;
14         return m_xPos;
15     }
16 };
17
18 int main()
19 {
20     return 0;
21 }
```

Note that I'm trying to change the attribute of the implicit object here.

```
PS C:\Users\mohammad.salman\OneDrive - Habib University\Documents\Course Stuff\Fall 2023\CS224 - OOP\
fc++ -Wextra -Wconversion -Wsign-conversion -std=c++14 .\5-access_specifiers.cpp
.\5-access_specifiers.cpp: In member function 'int Position::getXPos() const':
.\5-access_specifiers.cpp:13:18: error: assignment of member 'Position::m_xPos' in read-only object
    m_xPos = 54;
           ^~
PS C:\Users\mohammad.salman\OneDrive - Habib University\Documents\Course Stuff\Fall 2023\CS224 - OOP\
```

```
1  #include <iostream>
2
3  class Position
4  {
5  private:
6      int m_xPos;
7      int m_yPos;
8      int m_zPos;
9
10 public:
11     int getXPos(Position& obj) const
12     {
13         obj.m_xPos = 54;
14
15         m_xPos = 54;
16         return m_xPos;
17     }
18 };
19
20 int main()
21 {
22     Position p1;
23     Position p2;
24
25     p1.getXPos(p2);
26
27     return 0;
28 }
```

*I cannot change the attribute of the **implicit object** (which is **p1** in this e.g.).*

*But I could change the attribute of **another object** that was passed to this **const** method if I wanted to.*

*Over here, I'm changing the attribute of **obj** (or **p2**), which is fine.*

*If I wanted to prevent changes to **obj** (or **p2**) as well, I could pass it by **const reference**.*

Note for Students

(Recommended for all OOP Concepts)



Would recommend running the code we've seen on Python Tutor.

The visualization may help you understand.

Some things you may note are,

- The class definition by itself will not create any entity in memory – no space occupied.

- It is when you create an object does memory gets used up.

- Note what the attributes get initialized to.

- How can we change it?